GUÍA COMPLETA DE ESTUDIO Y PREPARACIÓN DE PARCIALES

Ingeniería del Software & Patrones de Diseño — UBA

1. Modern Software Engineering (David Farley) — Cap. 1, 2 y 3

Capítulo 1: La puesta en práctica de la ciencia. Farley sostiene que el desarrollo de software no es una disciplina de manufactura ni de producción en masa, sino un proceso continuo de exploración, descubrimiento y diseño creativo. Por esta razón, el éxito de los ingenieros y organizaciones radica en convertirse en **expertos en aprender**. Propone adoptar un método científico pragmático caracterizado por cuatro pasos: Caracterizar el problema, Hipotetizar, Predecir y Experimentar. Define operativamente la Ingeniería de Software como *'la aplicación de un método empírico y científico de encontrar maneras eficientes y económicas de solucionar problemas en software'*. Identifica dos competencias centrales: ser expertos en aprender y ser expertos en gestionar la complejidad.

Capítulo 2: Métodos Formales vs. Empirismo Pragmático. Farley analiza y critica los métodos formales de desarrollo que buscan demostrar matemáticamente la corrección del código. Afirma que en sistemas reales complejos, concurrentes e interactivos, la verificación matemática completa es inviable. Muestra el enfoque empírico mediante el ejemplo de SpaceX (ensayos destructivos para validar hipótesis) y rescata el trabajo pionero de Margaret Hamilton en el proyecto Apollo, centrado en aprender de los errores, la tolerancia a fallos y la ingeniería basada en la experiencia. Asimismo, critica severamente el modelo en Cascada (Waterfall), el cual asume erróneamente que los requerimientos y el diseño pueden fijarse de forma determinista antes de comenzar la construcción.

Capítulo 3: Elementos fundamentales del método ingenieril. Para evaluar el desempeño de un equipo se proponen dos métricas clave: **Stability (Estabilidad)**, que mide la calidad del software entregado (evaluada mediante el tiempo medio de recuperación / MTTR y la tasa de fallas en cambios / change failure rate), y **Throughput / Flow (Flujo / Velocidad)**, que mide la eficiencia de entrega (evaluada mediante el lead time de cambios y la frecuencia de despliegue). Farley enfatiza que velocidad y calidad no se oponen, sino que están correlacionadas. Define 5 conductas para aprender (trabajar iterativamente, usar feedback rápido, trabajar incrementalmente, ser experimental y empírico) y 5 pilares para gestionar la complejidad (modularidad, cohesión, separación de preocupaciones, abstracción/ocultamiento de información y manejo del acoplamiento).

PREGUNTA DE PARCIALITO #1.1: Explicar la diferencia entre las dos métricas que se proponen en el libro de Modern Software Engineering para evaluar la efectividad de un equipo de software. Explique además qué medidas específicas se rastrean para Stability y para Throughput.

Criterio de Respuesta y Modelo UBA: Estabilidad (Stability): Mide la calidad del software producido. Se evalúa rastreando dos medidas concretas: el tiempo medio de recuperación (**MTTR** - Mean Time To Recover) ante fallos y la tasa de fallos en cambios (**Change Failure Rate**).
Flujo (Throughput): Mide la velocidad y eficiencia en la entrega de valor. Se evalúa rastreando dos medidas concretas: el tiempo de entrega de cambios (**Lead Time for Changes**) y la frecuencia de despliegue (**Deployment Frequency**).
Enfoque UBA: Ambas métricas no se contraponen; los equipos de alto rendimiento logran alta estabilidad y alto flujo en simultáneo gracias al feedback rápido.

PREGUNTA DE PARCIALITO #1.2: ¿Cuál es el problema fundamental de los métodos formales de desarrollo de software y cuál es el problema de la metodología 'Cascada' (Waterfall) según David Farley?

Criterio de Respuesta y Modelo UBA: Métodos Formales: Intentan demostrar matemáticamente la corrección del software. El problema radica en que demostraciones matemáticas de sistemas reales complejos son más difíciles y costosas de escribir que el código mismo, y resultan inviables ante la concurrencia e interactividad.
Modelo en Cascada: Asume erróneamente que es posible definir todos los requerimientos y el diseño de manera determinista al inicio. Ignora que el desarrollo de software es un proceso continuo de exploración, descubrimiento y aprendizaje empírico.

---

2. Capítulos del GoF — State, Strategy y Template Method

State (GoF): Permite a un objeto alterar su comportamiento cuando cambia su estado interno, aparentando cambiar de clase. Participantes: Context (mantiene la instancia del estado actual), State (interfaz abstracta) y subclases ConcreteState. Reemplaza bloques monolíticos de condicionales (if/switch) por clases dedicadas por cada estado, haciendo las transiciones explícitas al rebinfiar una variable de estado. A diferencia de las tablas de transición (que se enfocan en mapear el flujo de cambios), el patrón State modela directamente el comportamiento asociado a cada estado. Si las subclases de estado no tienen variables de instancia, pueden compartirse como Singletons/Flyweights.

Strategy (GoF): Define una familia de algoritmos, encapsula cada uno y los hace intercambiables. Permite que el algoritmo varíe independientemente de los clientes que lo utilizan. Participantes: Context, Strategy y ConcreteStrategy.
• *Con Template Method:* Strategy varía el algoritmo completo mediante composición/delegación; Template Method usa herencia para variar partes del algoritmo.
• *Con State:* Comparten la estructura 'handle/body', pero Strategy es estático ('bind-once') mientras que State es dinámico en tiempo de ejecución.
• *Con Decorator:* Strategy cambia las entrañas/guts del objeto; Decorator cambia su piel/piel externa.

Template Method (GoF): Define el esqueleto de un algoritmo en una operación de la clase base, delegando ciertos pasos (operaciones primitivas o ganchos/hooks) a las subclases sin modificar la estructura general. Mantiene la invariante del flujo del algoritmo y evita duplicación de código mediante herencia.

PREGUNTA DE PARCIALITO #2.1: Compare el patrón Strategy con el patrón Template Method y con el patrón State. Indique diferencias en mecanismo, variabilidad y dinamismo.

Criterio de Respuesta y Modelo UBA: Strategy vs. Template Method: Strategy cambia el algoritmo completo utilizando composición y delegación. Template Method utiliza herencia en una clase base abstracta para fijar el esqueleto del algoritmo, permitiendo que las subclases redefinan únicamente ciertos pasos específicos (operaciones abstractas o hooks).
Strategy vs. State: Aunque tienen la misma estructura UML ('handle/body'), difieren en su intención y dinamismo. Strategy suele ser configurado por el cliente de forma estática o al inicio ('bind-once'). En State, el objeto Context cambia su estado interno de forma autónoma y dinámica a lo largo del ciclo de vida del sistema.

PREGUNTA DE PARCIALITO #2.2: ¿Cómo resuelve el patrón State la proliferación de condicionales en el código y en qué se diferencia el patrón State de una Tabla de Transición de Estados?

Criterio de Respuesta y Modelo UBA: Resolución de condicionales: Reemplaza las estructuras monolíticas `if/switch` dispersas en el Context por clases concretas dedicadas a cada estado. La lógica se encapsula polimórficamente en subclases de `State`, haciendo las transiciones explícitas mediante la reasignación del objeto estado.
Diferencia con Tablas de Transición: La tabla de transición es un enfoque guiado por datos que mapea insumos con estados futuros (útil si las transiciones cambian frecuentemente). Sin embargo, la tabla no encapsula el comportamiento ejecutable. El patrón State encapsula el comportamiento y las acciones de cada estado directamente en métodos polimórficos de objetos.

---

3. Respuesta sobre Computer Science de Alan Kay en Quora

Diagnóstico de la academia: Alan Kay denuncia que la educación universitaria en Ciencias de la Computación ha sufrido un **embrutecimiento deastroso ('dumbing down')**. Atribuye este deterioro a presiones comerciales y a la conversión de carreras universitarias en programas de formación vocacional/técnica, perdiendo la perspectiva histórica e intelectual de los fundadores.
Definición de 'Ciencia' y 'Ciencias de lo Artificial': Kay señala que los estudiantes actuales confunden ciencia con construir herramientas o escribir código. Basándose en Herb Simon, explica que la computación crea artefactos artificiales y luego estudia los fenómenos que emergen de ellos. Citando a Alan Perlis, define Computer Science como **'La ciencia de los procesos; todos los procesos'** (la ciencia de los sistemas dinámicos).
El rol de la Computadora y la Depuración: La computadora es un 'meta-medio' capaz de simular cualquier otro medio o sistema. Debido a la inmensa complejidad y dimensión temporal de los procesos, las soluciones no se demuestran formalmente de manera estática, sino que se **exploran y depuran (debugged)** en el espacio de procesos. En Xerox PARC, la ciencia de la computación (comprender) y la ingeniería (construir) retroalimentaban un espiral virtuoso.

PREGUNTA DE PARCIALITO #3.1: Alan Kay critica que los estudiantes actuales no pueden definir el término 'ciencia' en 'ciencias de la computación' más allá de algo parecido a la ingeniería de herramientas. ¿Qué significa 'ciencia' en este contexto según Alan Perlis y Alan Kay?

Criterio de Respuesta y Modelo UBA: Citando a Alan Perlis, definen Computer Science como **'La ciencia de los procesos (de todos los procesos)'**. No se limita a crear programas o herramientas de software aisladas, sino que es el estudio riguroso de la creación, modelado y comportamiento de sistemas complejos de procesos dinámicos que evolucionan en el tiempo (Ciencia de lo Artificial, según Herb Simon).

PREGUNTA DE PARCIALITO #3.2: según Alan Kay, ¿quiénes son los responsables del 'empobrecimiento' o embrutecimiento (dumbing down) de la Ciencia de la Computación y por qué se produjo este fenómeno?

Criterio de Respuesta y Modelo UBA: Los responsables son las instituciones de educación superior y la industria comercial. Se produjo al transformar la carrera de una disciplina científica y filosófica de investigación profunda sobre procesos a un sistema mercantilizado de capacitación vocacional orientada a la demanda laboral rápida de programadores de herramientas, perdiendo la perspectiva histórica y conceptual.

---

4. Patrón Observer y Active Variable (Smalltalk-80)

Observer (GoF): Define una dependencia uno-a-muchos entre objetos de modo que cuando el Subject cambia de estado, todos sus Observers son notificados y actualizados automáticamente.
• *Modelo Push:* El Subject envía datos detallados del cambio en la notificación (asume qué necesita el observador, reduciendo reusabilidad).
• *Modelo Pull:* El Subject envía únicamente una notificación mínima y los observadores consultan explícitamente lo necesario (mayor desacoplamiento).
• *ChangeManager:* Actúa como Mediator para gestionar relaciones complejas y evitar notificaciones redundantes en grafos de dependencias.
• *Aspects:* Permite registrar observadores para eventos o aspectos específicos de interés.

Active Variable (Smalltalk-80 — Messick & Beck): Es un mecanismo que reifica el acceso y asignación a variables mediante procedimientos auxiliares llamados **daemons**.
• *Get daemon:* Se invoca automáticamente cuando la variable es leída.
• *Set daemon:* Se invoca automáticamente cuando se le asigna un nuevo valor.
El compilador de Smalltalk modifica transparentemente el acceso a la variable convirtiéndolo en un mensaje enviado a un objeto de la jerarquía ActiveVariable. Se utiliza para construir interfaces reactivas (*'Dials and Gauges' /* medidores gráficos) y herramientas de depuración.

PREGUNTA DE PARCIALITO #4.1: Compare el patrón de diseño Observer (GoF) con el mecanismo de Active Variable en Smalltalk-80. Explique nivel de operación, acoplamiento y forma de notificación.

Criterio de Respuesta y Modelo UBA: Observer (GoF): Opera a nivel de **aplicación / diseño de clases**. La relación de suscripción es explícita entre clases `Subject` y `Observer`, mediante llamadas explícitas a `Attach()`, `Detach()` y `Notify()`.
Active Variable: Opera a nivel de **lenguaje / entorno / compilador**. Intercepta transparentemente las lecturas y escrituras de variables mediante **daemons** (`get` y `set`). No requiere que el cliente registre observadores mediante código de aplicación, sino que el compilador reifica el acceso a la variable.

PREGUNTA DE PARCIALITO #4.2: Explique la diferencia entre el Modelo Push y el Modelo Pull en el patrón Observer, y describa el rol de un ChangeManager.

Criterio de Respuesta y Modelo UBA: Push vs. Pull: En el modelo Push, el Subject envía información detallada del cambio dentro del mensaje de notificación. En el modelo Pull, el Subject envía una notificación mínima y es el Observer quien consulta (`GetState()`) únicamente los datos que necesita.
ChangeManager: Es un objeto Mediator que centraliza la lógica de notificación cuando las dependencias entre sujetos u observadores son complejas (ej. grafos acíclicos dirigidos / DAGChangeManager), reduciendo actualizaciones redundantes o inconsistentes.

---

5. Patrón Composite (GoF)

Intent y Participantes: Compone objetos en estructuras de árbol para representar jerarquías de parte-todo (*part-whole*). Permite a los clientes tratar de manera uniforme a objetos individuales (Leaf) y a composiciones de objetos (Composite). Participantes: Component, Leaf, Composite y Client.
El Dilema 'Safety' (Seguridad) vs. 'Transparency' (Transparencia):
• *Transparencia:* Define las operaciones de manejo de hijos (Add, Remove, GetChild) en la clase raíz Component. Ventaja: Uniformidad total para el cliente. Desventaja: Falta de seguridad de tipos en compilación (llamar a Add en una Leaf es un error de ejecución o requiere no-op).
• *Seguridad:* Declara el manejo de hijos únicamente en la clase Composite. Ventaja: Seguridad de tipos en compilación. Desventaja: Se pierde la uniformidad, obligando al cliente a realizar chequeos de tipo o cast (ej.

GetComposite()).
Postura GoF y de la Materia: Se prioriza la **Transparencia** sobre la seguridad para preservar el propósito esencial del patrón.

PREGUNTA DE PARCIALITO #5.1: Explique el dilema 'Safety' vs 'Transparency' del patrón Composite, indicando ventajas y desventajas de cada postura, y cuál es la postura adoptada por la materia y el GoF.

Criterio de Respuesta y Modelo UBA: Transparencia: Declara la interfaz de gestión de hijos en la raíz `Component`.
• *Ventaja:* Máxima uniformidad; el cliente trata hojas y compuestos mediante la misma interfaz.
• *Desventaja:* Inseguridad de tipos; una `Leaf` respondería a `Add` arrojando un error.
Seguridad: Declara la gestión de hijos únicamente en la clase `Composite`.
• *Ventaja:* Seguridad en compilación; imposible agregar un hijo a una hoja.
• *Desventaja:* Pérdida de interfaz uniforme; el cliente debe chequear tipos o usar `GetComposite()`.
Postura UBA/GoF: Se prioriza la **Transparencia** para sostener el principio rector del patrón de tratar a todos los elementos uniformemente.

PREGUNTA DE PARCIALITO #5.2: ¿Cómo se relaciona el patrón Composite con el patrón Decorator y el patrón Iterator?

Criterio de Respuesta y Modelo UBA: Con Decorator: Ambos se basan en la composición recursiva. Un Decorator puede verse como un Composite degenerado con un único componente, pero su propósito es agregar responsabilidades ('embellecer'), mientras que Composite se enfoca en la representación jerárquica de contenedores.
Con Iterator: Se utiliza para recorrer de forma polimórfica y transparente las estructuras de árbol compuestas por los nodos del Composite sin exponer su representación interna.

---

6. Cultures of Programming (Tomas Petricek) — Conclusiones

Concepto de Culturas de Programación: Petricek define las culturas como sistemas de práctica e ideales coherentes que han guiado el desarrollo de la programación durante 70 años. Identifica 5 culturas principales:
1. *Matemática:* El programa es una entidad formal; busca corrección probada matemáticamente.
2. *Hacker:* La programación es exploración directa, libertad creativa e interacción dinámica con la máquina.
3. *Ingenieril:* Ve la programación como resolución pragmática de problemas; foco en herramientas, testing y límites humanos.
4. *Gerencial:* Foco en procesos, especificaciones, estándares, control de proyectos y organización de equipos.
5. *Humanística:* El programa como medio para el pensamiento, la expresión y la democratización del acceso (Smalltalk, Dynabook).
Lecciones clave del Capítulo 7 (Conclusiones):
• *Pluralismo Esencial:* La programación es pluralista de forma estable; las 5 culturas coexisten en paralelo.
• *Trading Zones y Objetos Frontera (Boundary Objects):* Los conceptos (como los 'tests' o los 'objetos') evolucionan en zonas de intercambio entre culturas.
• *Cooptación:* Las ideas nacidas en una cultura suelen ser cooptadas por otra (ej. la cultura gerencial adaptó la programación estructurada y los objetos para estructurar equipos).
• *Conclusión Final ('Many things go'):* Inspirado en Feyerabend y Chang, concluye que no hay un único paradigma dogmático; la riqueza de la disciplina reside en el intercambio plural entre culturas.

PREGUNTA DE PARCIALITO #6.1: Mencione las 5 culturas de programación identificadas por Tomas Petricek y explique brevemente cómo concibe a un 'programa' la cultura Matemática vs la cultura Gerencial.

Criterio de Respuesta y Modelo UBA: Las 5 culturas son: **Matemática, Hacker, Ingenieril, Gerencial y Humanística**.
• **Cultura Matemática:** Ve al programa como una entidad formal definida en un lenguaje abstracto, cuya corrección debe ser demostrada mediante pruebas matemáticas.
• **Cultura Gerencial:** Le interesa menos la naturaleza interna del programa y lo ve como una herramienta para resolver problemas de negocio que debe construirse mediante procesos estructurados, estándares y metodologías que organizan equipos de trabajo.

PREGUNTA DE PARCIALITO #6.2: Explique los conceptos de 'Trading Zones' (Zonas de comercio) y 'Cooptación de conceptos' entre culturas de programación. De un ejemplo citado por Petricek.

Criterio de Respuesta y Modelo UBA: • **Trading Zones / Boundary Objects:** Son espacios de intercambio donde diferentes culturas colaboran en torno a un concepto técnico compartido (ej. 'testing' o 'excepciones') interpretándolo según sus propios ideales.
• **Cooptación:** Ocurre cuando una cultura adopta un concepto técnico creado por otra y lo transforma para sus propios fines. *Ejemplo:* La cultura Gerencial cooptó la programación estructurada y la orientación a objetos para definir la división de tareas en los equipos de desarrollo.

---

7. Patrones Wrappers: Decorator, Adapter y Proxy (GoF)

Decorator (GoF): Adjunta responsabilidades adicionales a un objeto de manera dinámica y transparente. Mantiene la **misma interfaz** del objeto envuelto ('cambia la piel del objeto').
Adapter (GoF): Convierte la interfaz de una clase en otra interfaz esperada por los clientes, resolviendo incompatibilidades. Proporciona una **interfaz diferente**. Variantes: Class Adapter (herencia múltiple), Object Adapter (composición), Pluggable Adapters (bloques en Smalltalk).
Proxy (GoF): Proporciona un sustituto o representante (surrogate) para controlar el acceso al objeto real. Mantiene la **misma interfaz**.
• *Variantes:* Virtual Proxy (carga diferida/lazy instantiation), Remote Proxy (representante local en otra red), Protection Proxy (control de permisos), Smart Reference (recuento de referencias/housekeeping).
• *Proxy en Smalltalk:* Se implementa definiendo una clase con superclase nil y redefiniendo doesNotUnderstand: aMessage para interceptar mensajes y reenviarlos mediante perform:withArguments:.
Cuadro Comparativo:
• *Adapter:* Interfaz Diferente | Propósito: Resolver incompatibilidad.
• *Decorator:* Misma Interfaz (Ampliada) | Propósito: Añadir responsabilidades dinámicamente.
• *Proxy:* Misma Interfaz | Propósito: Controlar acceso / indirection.

PREGUNTA DE PARCIALITO #7.1: Compare los patrones Proxy, Adapter y Decorator en cuanto a su interfaz con respecto al objeto envuelto y su intención. Muestre un cuadro comparativo simple.

Criterio de Respuesta y Modelo UBA: • **Adapter:** Proporciona una interfaz **diferente** para adaptar una clase incompatible a lo que el cliente espera.
• **Decorator:** Mantiene la **misma interfaz** (conforme al Component), pero la **enriquece/amplía** dinámicamente agregando responsabilidades ('cambia la piel').
• **Proxy:** Mantiene exactamente la **misma interfaz** que el objeto real para controlar, diferir o proteger el acceso al mismo.

PREGUNTA DE PARCIALITO #7.2: ¿Cómo se diferencia en Smalltalk, a nivel implementación, un Proxy polimórfico de uno no polimórfico? Explique el uso de superclase nil y doesNotUnderstand:.

Criterio de Respuesta y Modelo UBA: Un **Proxy polimórfico genérico** en Smalltalk se define creando una clase cuya superclase es `nil` (para no heredar ningún método de `Object`). De este modo, cualquier mensaje enviado al Proxy fallará en la búsqueda básica e invocará el método `doesNotUnderstand: aMessage`. El Proxy reintercepta dicho mensaje y se lo reenvía de forma transparente al objeto real mediante `realSubject perform: aMessage selector withArguments: aMessage arguments`.

---

8. Patrón Visitor (GoF)

Intent y Participantes: Representa una operación a realizar sobre los elementos de una estructura de objetos sin modificar las clases de dichos elementos. Participantes: Visitor, ConcreteVisitor, Element, ConcreteElement y ObjectStructure.
Mecanismo Clave — Double Dispatch (Doble Despacho): La operación que se ejecuta depende del tipo de dos receptores: el tipo del Visitor concreto y el tipo del Elemento concreto. La llamada arranca con element.Accept(visitor), e internamente el elemento hace un callback llamando a visitor.VisitConcreteElement(this).
Dos Patrones / Mecanismos Subyacentes (Pregunta típica UBA):
1. *Double Dispatch:* Mecanismo para resolver polimórficamente el método a ejecutar según el par (Visitor, Element).
2. *Composite (o Recorrido en Árbol):* Estructura jerárquica (como un Árbol de Sintaxis Abstracta - AST) sobre la cual itera el Visitor.
Trade-offs: Es fácil agregar nuevas **operaciones** (creando una subclase de Visitor sin tocar las clases de elementos). Es difícil agregar nuevos **elementos** (obliga a modificar la interfaz `Visitor` y actualizar todas sus subclases).

PREGUNTA DE PARCIALITO #8.1: ¿Qué dos patrones o mecanismos de diseño subyacen en toda solución modelada con un patrón Visitor? Explique brevemente en qué parte de la estructura se utiliza cada uno.

Criterio de Respuesta y Modelo UBA: 1. **Doble Despacho (Double Dispatch):** Es el mecanismo por el cual la operación ejecutada depende de la combinación del tipo del Visitor y del Elemento. Se utiliza en el método `element.Accept(visitor)`, donde el elemento realiza un callback ejecutando `visitor.VisitConcreteElement(this)`.
2. **Composite (o Recorrido Jerárquico):** Es la estructura de objetos sobre la cual opera el Visitor (ej. los nodos de un árbol AST de un compilador), permitiendo recorrer recursivamente la jerarquía para aplicar la operación.

PREGUNTA DE PARCIALITO #8.2: Explique las ventajas y desventajas (trade-offs) del patrón Visitor respecto a la extensibilidad del sistema.

Criterio de Respuesta y Modelo UBA: • **Ventaja (Fácil agregar Operaciones):** Permite definir nuevas operaciones sobre la estructura de objetos simplemente agregando una nueva subclase de `Visitor` (ej. `TypeCheckingVisitor`, `CodeGeneratingVisitor`) sin recompilar ni alterar las clases de los elementos.
• **Desventaja (Difícil agregar Elementos):** Si se agrega una nueva clase de `ConcreteElement`, se debe modificar la interfaz abstracta `Visitor` para añadir el nuevo método `Visit...`, obligando a redefinirlo e implementarlo en **todas** las subclases concretas de Visitor existentes.